

```
1  #include <windows.h>
2  # include<stdio.h> // for file I/O
3
4  #include <d3d11.h>
5
6  #pragma comment (lib,"d3d11.lib")
7
8  #define WIN_WIDTH 800
9  #define WIN_HEIGHT 600
10
11 // global function declarations
12 LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
13
14 // global varibale declarations
15 FILE *gpFile = NULL;
16 char gszLogFileName[] = "Log.txt";
17
18 HWND ghwnd = NULL;
19
20 DWORD dwStyle;
21 WINDOWPLACEMENT wpPrev = { sizeof(WINDOWPLACEMENT) };
22
23 bool gbActiveWindow = false;
24 bool gbEscapeKeyIsPressed = false;
25 bool gbFullscreen = false;
26
27 float gClearColor[4]; // RGBA
28 IDXGISwapChain *gpIDXGISwapChain = NULL;
29 ID3D11Device *gpID3D11Device = NULL;
30 ID3D11DeviceContext *gpID3D11DeviceContext = NULL;
31 ID3D11RenderTargetView *gpID3D11RenderTargetView = NULL;
32
33 // WinMain
34 int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpszCmdLine, int iCmdShow)
35 {
36     // function declarations
37     HRESULT initialize(void);
38     void uninitialize(void);
39     void display(void);
40
41     // variable declarations
42     WNDCLASSEX wndclass;
43     HWND hwnd;
44     MSG msg;
45     TCHAR szClassName[] = TEXT("Direct3D11");
46     bool bDone = false;
47
48     // code
49     // create log file
50     if (fopen_s(&gpFile, gszLogFileName, "w") != 0)
51     {
```

```
52     MessageBox(NULL, TEXT("Log File Can Not Be Created\nExiting ..."), TEXT("Error"), MB_OK | MB_TOPMOST | MB_ICONSTOP);
53     exit(0);
54 }
55 else
56 {
57     fprintf_s(gpFile, "Log File Is Successfully Opened.\n");
58     fclose(gpFile);
59 }
60
61 // initialize WNDCLASSEX structure
62 wndclass.cbSize = sizeof(WNDCLASSEX);
63 wndclass.style = CS_HREDRAW | CS_VREDRAW | CS_OWNDC;
64 wndclass.cbClsExtra = 0;
65 wndclass.cbWndExtra = 0;
66 wndclass.lpfnWndProc = WndProc;
67 wndclass.hbrBackground = (HBRUSH)GetStockObject(BLACK_BRUSH);
68 wndclass.hInstance = hInstance;
69 wndclass.hIcon = LoadIcon(NULL, IDI_APPLICATION);
70 wndclass.hCursor = LoadCursor(NULL, IDC_ARROW);
71 wndclass.lpszClassName = szClassName;
72 wndclass.lpszMenuName = NULL;
73 wndclass.hIconSm = LoadIcon(NULL, IDI_APPLICATION);
74
75 // register WNDCLASSEX structure
76 RegisterClassEx(&wndclass);
77
78 // create window
79 hwnd = CreateWindow(szClassName,
80     TEXT("Direct3D11 Window"),
81     WS_OVERLAPPEDWINDOW,
82     100,
83     100,
84     WIN_WIDTH,
85     WIN_HEIGHT,
86     NULL,
87     NULL,
88     hInstance,
89     NULL);
90
91 ghwnd = hwnd;
92
93 ShowWindow(hwnd, iCmdShow);
94 SetForegroundWindow(hwnd);
95 SetFocus(hwnd);
96
97 // initialize D3D
98 HRESULT hr;
99 hr = initialize();
100 if (FAILED(hr))
101 {
102     fopen_s(&gpFile, gszLogFileName, "a+");
```

```
103     fprintf_s(gpFile, "initialize() Failed. Exiting Now ...\n");
104     fclose(gpFile);
105     DestroyWindow(hwnd);
106     hwnd = NULL;
107 }
108 else
109 {
110     fopen_s(&gpFile, gszLogFileName, "a+");
111     fprintf_s(gpFile, "initialize() Succeeded.\n");
112     fclose(gpFile);
113 }
114
115 // message loop
116 while (bDone == false)
117 {
118     if (PeekMessage(&msg, NULL, 0, 0, PM_REMOVE))
119     {
120         if (msg.message == WM_QUIT)
121             bDone = true;
122         else
123         {
124             TranslateMessage(&msg);
125             DispatchMessage(&msg);
126         }
127     }
128     else
129     {
130         // render
131         display();
132
133         if (gbActiveWindow == true)
134         {
135             if (gbEscapeKeyIsPressed == true)
136                 bDone = true;
137         }
138     }
139 }
140
141 // clean-up
142 uninitialized();
143
144 return((int)msg.wParam);
145 }
146
147 LRESULT CALLBACK WndProc(HWND hwnd, UINT iMsg, WPARAM wParam, LPARAM lParam)
148 {
149     // function declarations
150     HRESULT resize(int, int);
151     void ToggleFullscreen(void);
152     void uninitialized(void);
153
154     // variable declaratrions
```

```
155     HRESULT hr;
156
157     // code
158     switch (iMsg)
159     {
160     case WM_ACTIVATE:
161         if (HIWORD(wParam) == 0) // if 0, window is active
162             gbActiveWindow = true;
163         else // if non-zero, window is not active
164             gbActiveWindow = false;
165         break;
166     case WM_ERASEBKGND:
167         return(0);
168     case WM_SIZE:
169         if (gpID3D11DeviceContext)
170         {
171             hr = resize(LOWORD(lParam), HIWORD(lParam));
172             if (FAILED(hr))
173             {
174                 fopen_s(&gpFile, gszLogFileName, "a+");
175                 fprintf_s(gpFile, "resize() Failed.\n");
176                 fclose(gpFile);
177                 return(hr);
178             }
179             else
180             {
181                 fopen_s(&gpFile, gszLogFileName, "a+");
182                 fprintf_s(gpFile, "resize() Succeeded.\n");
183                 fclose(gpFile);
184             }
185         }
186         break;
187     case WM_KEYDOWN:
188         switch (wParam)
189         {
190         case VK_ESCAPE: // case 27
191             if (gbEscapeKeyIsPressed == false)
192                 gbEscapeKeyIsPressed = true;
193             break;
194         case 0x46: // 'F' or 'f'
195             if (gbFullscreen == false)
196             {
197                 ToggleFullscreen();
198                 gbFullscreen = true;
199             }
200             else
201             {
202                 ToggleFullscreen();
203                 gbFullscreen = false;
204             }
205             break;
206         default:
```

```
207         break;
208     }
209     break;
210     case WM_LBUTTONDOWN:
211         break;
212     case WM_CLOSE:
213         uninitialized();
214         break;
215     case WM_DESTROY:
216         PostQuitMessage(0);
217         break;
218     default:
219         break;
220 }
221 return(DefWindowProc(hwnd, iMsg, wParam, lParam));
222 }
223
224 void ToggleFullscreen(void)
225 {
226     // variable declarations
227     MONITORINFO mi;
228
229     // code
230     if (gbFullscreen == false)
231     {
232         dwStyle = GetWindowLong(ghwnd, GWL_STYLE);
233         if (dwStyle & WS_OVERLAPPEDWINDOW)
234         {
235             mi = { sizeof(MONITORINFO) };
236             if (GetWindowPlacement(ghwnd, &wpPrev) && GetMonitorInfo
237                 (MonitorFromWindow(ghwnd, MONITORINFOF_PRIMARY), &mi))
238             {
239                 SetWindowLong(ghwnd, GWL_STYLE, dwStyle & ~WS_OVERLAPPEDWINDOW);
240                 SetWindowPos(ghwnd, HWND_TOP, mi.rcMonitor.left,
241                     mi.rcMonitor.top, mi.rcMonitor.right - mi.rcMonitor.left,
242                     mi.rcMonitor.bottom - mi.rcMonitor.top, SWP_NOZORDER |
243                     SWP_FRAMECHANGED);
244             }
245         }
246         ShowCursor(FALSE);
247     }
248     else
249     {
250         SetWindowLong(ghwnd, GWL_STYLE, dwStyle | WS_OVERLAPPEDWINDOW);
251         SetWindowPlacement(ghwnd, &wpPrev);
252         SetWindowPos(ghwnd, HWND_TOP, 0, 0, 0, 0, SWP_NOMOVE | SWP_NOSIZE |
253             SWP_NOOWNERZORDER | SWP_NOZORDER | SWP_FRAMECHANGED);
254         ShowCursor(TRUE);
255     }
256 }
```

```

254 HRESULT initialize(void)
255 {
256     // function declarations
257     void uninitialize(void);
258     HRESULT resize(int, int);
259
260     // variable declarations
261     HRESULT hr;
262     D3D_DRIVER_TYPE d3dDriverType;
263     D3D_DRIVER_TYPE d3dDriverTypes[] =
264         { D3D_DRIVER_TYPE_HARDWARE, D3D_DRIVER_TYPE_WARP, D3D_DRIVER_TYPE_REFERENCE,
265         };
266     D3D_FEATURE_LEVEL d3dFeatureLevel_required = D3D_FEATURE_LEVEL_11_0;
267     D3D_FEATURE_LEVEL d3dFeatureLevel_acquired = D3D_FEATURE_LEVEL_10_0; //
268         default, lowest
269     UINT createDeviceFlags = 0;
270     UINT numDriverTypes = 0;
271     UINT numFeatureLevels = 1; // based upon d3dFeatureLevel_required
272
273     // code
274     numDriverTypes = sizeof(d3dDriverTypes) / sizeof(d3dDriverTypes[0]); //
275         calculating size of array
276
277     DXGI_SWAP_CHAIN_DESC dxgiSwapChainDesc;
278     ZeroMemory((void *)&dxgiSwapChainDesc, sizeof(DXGI_SWAP_CHAIN_DESC));
279     dxgiSwapChainDesc.BufferCount = 1;
280     dxgiSwapChainDesc.BufferDesc.Width = WIN_WIDTH;
281     dxgiSwapChainDesc.BufferDesc.Height = WIN_HEIGHT;
282     dxgiSwapChainDesc.BufferDesc.Format = DXGI_FORMAT_R8G8B8A8_UNORM;
283     dxgiSwapChainDesc.BufferDesc.RefreshRate.Numerator = 60;
284     dxgiSwapChainDesc.BufferDesc.RefreshRate.Denominator = 1;
285     dxgiSwapChainDesc.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
286     dxgiSwapChainDesc.OutputWindow = ghwnd;
287     dxgiSwapChainDesc.SampleDesc.Count = 1;
288     dxgiSwapChainDesc.SampleDesc.Quality = 0;
289     dxgiSwapChainDesc.Windowed = TRUE;
290
291     for (UINT driverTypeIndex = 0; driverTypeIndex < numDriverTypes;
292         driverTypeIndex++)
293     {
294         d3dDriverType = d3dDriverTypes[driverTypeIndex];
295         hr = D3D11CreateDeviceAndSwapChain(
296             NULL, // Adapter
297             d3dDriverType, // Driver Type
298             NULL, // Software
299             createDeviceFlags, // Flags
300             &d3dFeatureLevel_required, // Feature Levels
301             numFeatureLevels, // Num Feature Levels
302             D3D11_SDK_VERSION, // SDK Version
303             &dxgiSwapChainDesc, // Swap Chain Desc
304             &gpIDXGISwapChain, // Swap Chain
305             &gpID3D11Device, // Device

```

```
301         &d3dFeatureLevel_acquired,    // Feature Level
302         &gpID3D11DeviceContext);      // Device Context
303     if (SUCCEEDED(hr))
304         break;
305 }
306 if (FAILED(hr))
307 {
308     fopen_s(&gpFile, gszLogFileName, "a+");
309     fprintf_s(gpFile, "D3D11CreateDeviceAndSwapChain() Failed.\n");
310     fclose(gpFile);
311     return(hr);
312 }
313 else
314 {
315     fopen_s(&gpFile, gszLogFileName, "a+");
316     fprintf_s(gpFile, "D3D11CreateDeviceAndSwapChain() Succeeded.\n");
317     fprintf_s(gpFile, "The Chosen Driver Is Of ");
318     if (d3dDriverType == D3D_DRIVER_TYPE_HARDWARE)
319     {
320         fprintf_s(gpFile, "Hardware Type.\n");
321     }
322     else if (d3dDriverType == D3D_DRIVER_TYPE_WARP)
323     {
324         fprintf_s(gpFile, "Warp Type.\n");
325     }
326     else if (d3dDriverType == D3D_DRIVER_TYPE_REFERENCE)
327     {
328         fprintf_s(gpFile, "Reference Type.\n");
329     }
330     else
331     {
332         fprintf_s(gpFile, "Unknown Type.\n");
333     }
334
335     fprintf_s(gpFile, "The Supported Highest Feature Level Is ");
336     if (d3dFeatureLevel_acquired == D3D_FEATURE_LEVEL_11_0)
337     {
338         fprintf_s(gpFile, "11.0\n");
339     }
340     else if (d3dFeatureLevel_acquired == D3D_FEATURE_LEVEL_10_1)
341     {
342         fprintf_s(gpFile, "10.1\n");
343     }
344     else if (d3dFeatureLevel_acquired == D3D_FEATURE_LEVEL_10_0)
345     {
346         fprintf_s(gpFile, "10.0\n");
347     }
348     else
349     {
350         fprintf_s(gpFile, "Unknown.\n");
351     }
352 }
```

```
353     fclose(gpFile);
354 }
355
356 // d3d clear color ( blue )
357 gClearColor[0] = 0.0f;
358 gClearColor[1] = 0.0f;
359 gClearColor[2] = 1.0f;
360 gClearColor[3] = 1.0f;
361
362 // call resize for first time
363 hr = resize(WIN_WIDTH, WIN_HEIGHT);
364 if (FAILED(hr))
365 {
366     fopen_s(&gpFile, gszLogFileName, "a+");
367     fprintf_s(gpFile, "resize() Failed.\n");
368     fclose(gpFile);
369     return(hr);
370 }
371 else
372 {
373     fopen_s(&gpFile, gszLogFileName, "a+");
374     fprintf_s(gpFile, "resize() Succeeded.\n");
375     fclose(gpFile);
376 }
377
378 return(S_OK);
379 }
380
381 HRESULT resize(int width, int height)
382 {
383     // code
384     HRESULT hr = S_OK;
385
386     // free any size-dependant resources
387     if (gpID3D11RenderTargetView)
388     {
389         gpID3D11RenderTargetView->Release();
390         gpID3D11RenderTargetView = NULL;
391     }
392
393     // resize swap chain buffers accordingly
394     gpIDXGISwapChain->ResizeBuffers(1, width, height, DXGI_FORMAT_R8G8B8A8_UNORM, 0);
395
396     // again get back buffer from swap chain
397     ID3D11Texture2D *pID3D11Texture2D_BackBuffer;
398     gpIDXGISwapChain->GetBuffer(0, __uuidof(ID3D11Texture2D), (LPVOID*)&pID3D11Texture2D_BackBuffer);
399
400     // again get render target view from d3d11 device using above back buffer
401     hr = gpID3D11Device->CreateRenderTargetView(pID3D11Texture2D_BackBuffer, NULL, &gpID3D11RenderTargetView);
```

```
402     if (FAILED(hr))
403     {
404         fopen_s(&gpFile, gszLogFileName, "a+");
405         fprintf_s(gpFile, "ID3D11Device::CreateRenderTargetView() Failed.\n");
406         fclose(gpFile);
407         return(hr);
408     }
409     else
410     {
411         fopen_s(&gpFile, gszLogFileName, "a+");
412         fprintf_s(gpFile, "ID3D11Device::CreateRenderTargetView() Succeeded.\n");
413         fclose(gpFile);
414     }
415     pID3D11Texture2D_BackBuffer->Release();
416     pID3D11Texture2D_BackBuffer = NULL;
417
418     // set render target view as render target
419     gpID3D11DeviceContext->OMSetRenderTargets(1, &gpID3D11RenderTargetView,
420         NULL);
421
422     // Set viewport
423     D3D11_VIEWPORT d3dViewPort;
424     d3dViewPort.TopLeftX = 0;
425     d3dViewPort.TopLeftY = 0;
426     d3dViewPort.Width = (float)width;
427     d3dViewPort.Height = (float)height;
428     d3dViewPort.MinDepth = 0.0f;
429     d3dViewPort.MaxDepth = 1.0f;
430     gpID3D11DeviceContext->RSSetViewports(1, &d3dViewPort);
431
432     return(hr);
433 }
434
435 void display(void)
436 {
437     // code
438     // clear render target view to a chosen color
439     gpID3D11DeviceContext->ClearRenderTargetView(gpID3D11RenderTargetView,
440         gClearColor);
441
442     // switch between front & back buffers
443     gpIDXGISwapChain->Present(0, 0);
444 }
445
446 void uninitialized(void)
447 {
448     // code
449     if (gpID3D11RenderTargetView)
450     {
451         gpID3D11RenderTargetView->Release();
452         gpID3D11RenderTargetView = NULL;
453     }
454 }
```

```
452
453     if (gpIDXGISwapChain)
454     {
455         gpIDXGISwapChain->Release();
456         gpIDXGISwapChain = NULL;
457     }
458
459     if (gpID3D11DeviceContext)
460     {
461         gpID3D11DeviceContext->Release();
462         gpID3D11DeviceContext = NULL;
463     }
464
465     if (gpID3D11Device)
466     {
467         gpID3D11Device->Release();
468         gpID3D11Device = NULL;
469     }
470
471     if (gpFile)
472     {
473         fopen_s(&gpFile, gszLogFileName, "a+");
474         fprintf_s(gpFile, "uninitialize() Succeeded\n");
475         fprintf_s(gpFile, "Log File Is Successfully Closed.\n");
476         fclose(gpFile);
477     }
478 }
479
```